

IBM - BUILDING RAG APPLICATIONS

WHY RAG ?

RAG solves the fundamental limitations of standalone Large Language Models.

- **Prevents Hallucinations:** Constrains the LLM to generate responses using retrieved facts rather than predicting plausible-sounding text.
- **Avoids Knowledge Cutoffs:** Provides real-time access to live data, APIs, and fresh documents without needing constant internet updates.
- **Eliminates Retraining Costs:** Updates knowledge instantly via vector databases, avoiding millions of dollars in model fine-tuning or full pre-training.
- **Accesses Private Data:** Allows public LLMs to securely answer questions using proprietary internal files, private company policies, or customer data
- **Enables Fact-Checking:** Provides source attribution and exact document chunk IDs so users can verify where the answer came from.

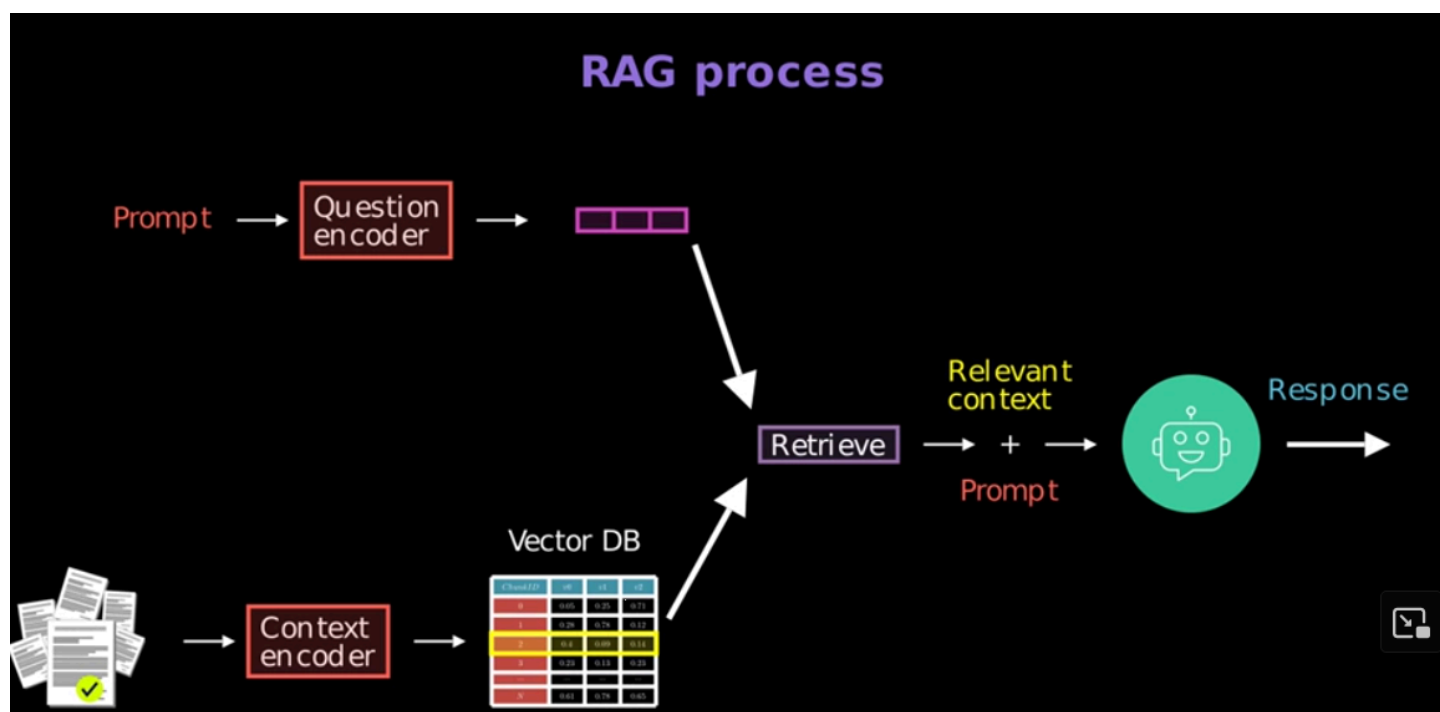
WHAT IS RAG ?

Retrieval-Augmented Generation (RAG) is an AI framework that optimizes the output of Large Language Models (LLMs) by integrating specific external domain knowledge or organizational databases without retraining the model.

While pre-trained LLMs perform well on general tasks, they struggle with specialized, domain-specific knowledge (such as confidential company policies) and can generate inaccurate responses. RAG addresses this by combining retrieved information with the LLM's natural language capabilities to generate precise, grounded answers.

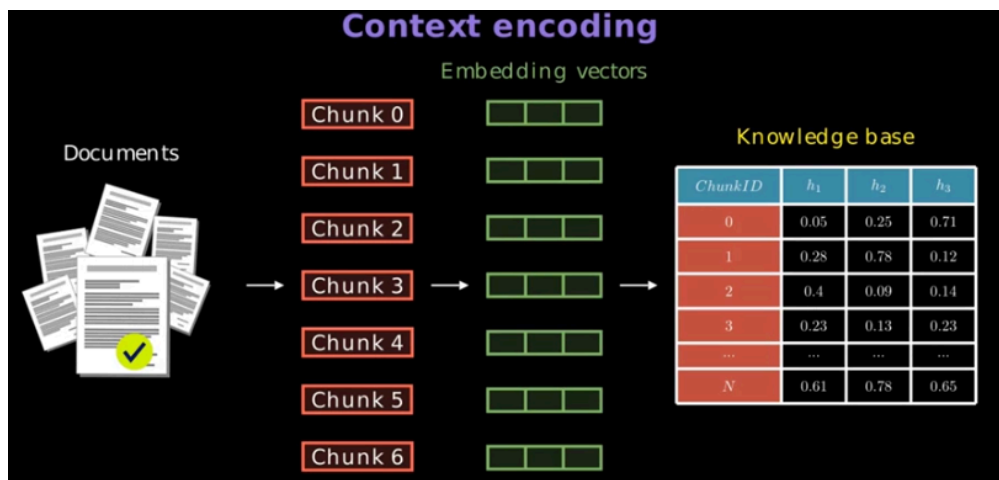
Key Components

- **Retriever:** The core of RAG, responsible for finding relevant information in the external knowledge base.
- **Generator:** Functions as the chatbot to create natural language responses using the retrieved context.



Step-by-Step RAG Process

Text & Document Embedding



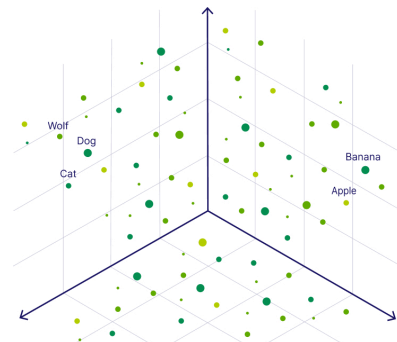
- **Document Chunking:** Large documents are split into smaller, manageable text chunks to enable targeted retrieval.

○

Encoding Chunks: Text chunks are transformed into **high-dimensional vectors** using a pre-trained token embedding model (e.g., BERT or GPT) and **vector averaging** across tokens.

WHAT ARE HIGH DIMENSIONAL VECTOR EMBEDDING ?

A high-dimensional vector embedding turns complex data like text, images, or audio into a long list of numbers. Each number acts as a feature in a large geometric space. Similar items sit close together in this space, letting computers measure meaning, find matches, and run AI tasks. Meilisearch v3



WHAT IS VECTOR AVERAGING AND WHY ITS DONE.

Vector averaging (or mean pooling) takes the individual vector embeddings of every token in a text chunk or prompt and averages them together to produce a single mathematical vector.

TXT

WHAT It Is

When text is processed, an encoder converts each individual word or token into its own high-dimensional vector. Vector averaging sums up all those individual token vectors along every dimension and divides by the total number of tokens. T X T + 1

$$V_{\text{chunk}} = \frac{v_1 + v_2 + \dots + v_N}{N}$$

The output is a single vector of the exact same size that represents the chunk or prompt as a whole. T X T

WHY It Is Used

1. **Creates a Single Representation:** Distance metrics (like Dot Product or Cosine Similarity) require comparing two single vectors. Averaging condenses a multi-word prompt or document chunk into one vector for easy comparison. T X T + 1
2. **Compresses Meaning Concisely:** It captures the overall semantic context and main topic of the prompt or chunk in a compact form. T X T
3. **Extremely Lightweight & Fast:** Computing a mathematical average is computationally cheap, making it easy to generate vectors quickly for fast database lookups. T X T

Limitations

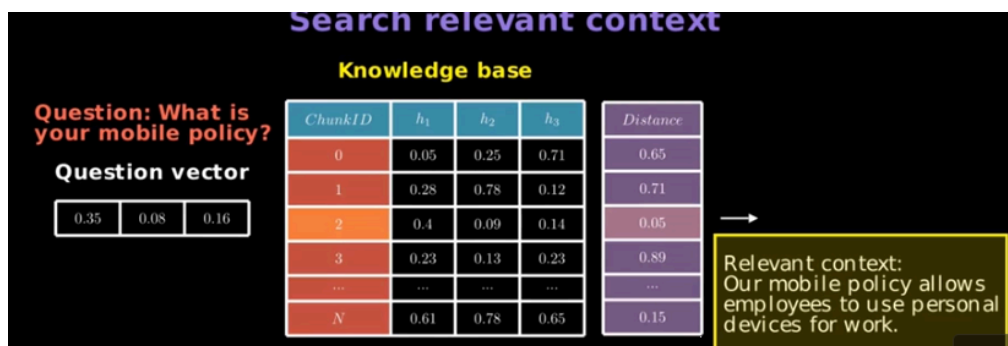
- **Loss of Word Order:** Averaging ignores word sequence and syntax ("dog bites man" vs. "man bites dog" yield the same average).
- **Information Dilution:** In long text chunks, averaging can wash out specific key details, which is why breaking documents into small chunks before embedding is necessary. T X T

- **Indexing:** Vector representations are saved in a vector database using a **Chunk ID** as the key.

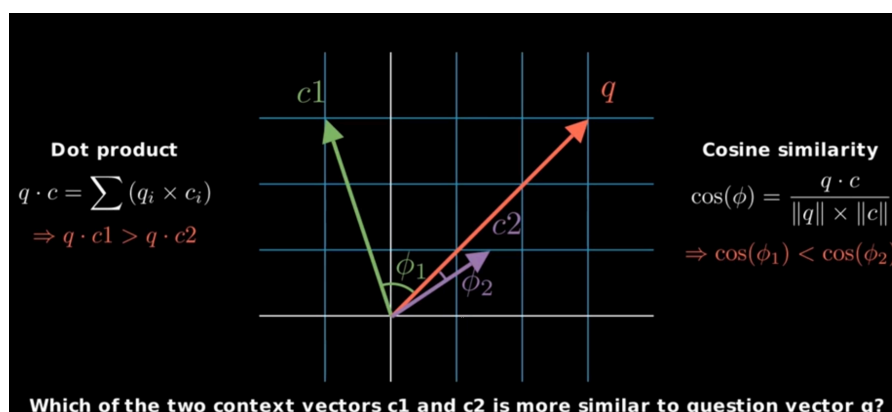
Prompt Encoding

- The user's input prompt is tokenized and transformed into a single high-dimensional vector q using token embedding models and vector averaging.

Retrieval via Vector Distance Metrics



- The prompt vector q is compared against stored context vectors (c_1, c_2) in the knowledge base.
- **Distance Metrics:** Calculates similarity using metrics like **Dot Product** (prioritizes vector magnitude and overall alignment) or **Cosine Distance** (focuses on angular direction).



- The top K closest context vectors (typically 3–5 chunk IDs like chunks 6, 2, and 0) are retrieved.

Augmented Query Creation

- The raw text associated with the retrieved vector chunk IDs is combined with the user's original query into a single augmented query.

Model Generation

- The language model uses the newly created augmented query to generate an accurate, domain-specific response grounded in the provided knowledge base content.